BANCO DE DADOS · AULA 06 MSSQL · T-SQL

Cursors e FETCH

Criação e Manipulação de Cursors · Navegação de Dados, Desempenho e Alternativas Set-Based.

ROW-BY-ROW | SET VS PROCEDURAL | PERFORMANCE & ALTERNATIVAS

CursorIteracao.sql — SSMS

› Cursor declarativo em ação

```
-- Cursor declarativo em ação
DECLARE    CURSOR FOR
SELECT
FROM
WHERE         'ATIVO'
OPEN
FETCH NEXT FROM    INTO @Id  @Saldo
WHILE @@FETCH_STATUS 0
BEGIN
    PRINT @Id
    FETCH NEXT FROM    INTO @Id  @Saldo
END
CLOSE     DEALLOCATE
```

EM PRODUÇÃO ▶ Aula 06 · FETCH

SUMÁRIO

Agenda da Aula

Dois blocos cobrindo sintaxe prática e decisão arquitetural sobre cursors.

DURAÇÃO TOTAL

100′

6.1

Criação e Manipulação de Cursors em SQL Server

- ›O que são cursors e quando surgiram
- ›Tipos de cursor (STATIC, KEYSET, DYNAMIC, FAST_FORWARD)
- ›Anatomia do DECLARE CURSOR
- ›Ciclo OPEN/FETCH/CLOSE/DEALLOCATE
- ›@@FETCH_STATUS
- ›Parâmetros de scroll (FIRST, LAST, PRIOR, NEXT, ABSOLUTE, RELATIVE)
- ›Cursors em stored procedures
- ›Exemplo prático completo

~50 min

6.2

Navegação de Dados, Desempenho e Alternativas ao Uso de Cursores

- ›Por que cursors são problemáticos
- ›Custos de CPU, memória e locks
- ›WHILE loop vs cursor
- ›Alternativas set-based (CTEs recursivas, window functions, MERGE)
- ›Alternativas modernas (CLR, Service Broker, JSON)
- ›Casos legítimos de uso
- ›Checklist profissional de quando usar/evitar

~50 min

Total estimado: 100 minutos · 18 slides · 12+ exemplos de código T-SQL reproduzíveis

lab_session.sql

BLOCO 6.1 / 2

6.1

~ 50 MIN · 8 TÓPICOS

CHAPTER

Criação e Manipulação de Cursors em SQL Server

Do conceito à sintaxe — tipos, anatomia, ciclo de vida e padrões de uso.

01 → O que são cursors

02 → Tipos (STATIC, KEYSET, DYNAMIC, FAST_FORWARD)

03 → DECLARE CURSOR

04 → OPEN/FETCH/CLOSE/DEALLOCATE

05 → @@FETCH_STATUS

06 → Scroll parameters

07 → SPs com cursor

08 → Exemplo completo

CONCEITO FUNDAMENTAL

O que são Cursors?

Cursor é um mecanismo que permite iterar sobre um result set linha por linha — o oposto do paradigma set-based do SQL. Enquanto um `SELECT` comum retorna todas as linhas de uma vez para o cliente, um cursor permite processar uma linha por vez dentro do servidor, com controle fino de posição.

Cursors existem porque herdam o modelo procedural de linguagens como C, Pascal e COBOL — onde loops são naturais. Em T-SQL, no entanto, são o anti-pattern nº1: o otimizador não consegue paralelizar operações linha-a-linha, e o overhead por linha é alto.

CARACTERÍSTICAS

- ›Ponteiro para uma posição em um result set
- ›Permite avançar (e em alguns tipos, retroceder)

CICLO DE VIDA DE UM CURSOR

Seis etapas obrigatórias — ordem deve ser respeitada.

1. **DECLARE**
   Define o cursor: consulta base + tipo + opções
2. **OPEN**
   Executa o SELECT e popula o result set (memória ou tempdb)
3. **FETCH**
   Lê a próxima linha e popula variáveis locais via INTO
4. **PROCESS**
   Lógica de negócio linha-a-linha (PRINT, INSERT, UPDATE...)
5. **CLOSE**
   Libera locks e result set, mantém a definição (re-OPEN possível)
6. **DEALLOCATE**
   Remove o cursor completamente da memória — etapa final mandatória

ⓘ Pular CLOSE ou DEALLOCATE causa vazamento de recursos e locks eternos — sempre envolva em TRY/CATCH.

COMPARATIVO

Tipos de Cursor no SQL Server

Cinco tipos com trade-offs distintos entre performance, funcionalidade e consistência.

✓ OK ⚠ WARN ✗ NO

Característica	FAST_FORWARDrecomendado	FORWARD_ONLYbásico	STATICsnapshot	KEYSETmembership fixo	DYNAMICevitar
Sinônimo	READ_ONLYFORWARD_ONLY	FORWARD_ONLYSTATIC	INSENSITIVE	—	—
Performance	✓✓ Melhor	✓ Boa	⚠ Média	⚠ Média	✗ Pior
Direção	Só frente	Só frente	Bidirecional	Bidirecional	Bidirecional
Snap-shot?	Não	Não	Sim (em tempdb)	Parcial	Não
Vê mudanças?	Não (read-only)	Não	Não (snapshot)	Sim (inserções não)	Sim (todas)
Muda posição?	Sim (outros alteram)	Sim	Não (snapshot fixo)	Não (membership fixo)	Sim (membership varia)
Memória	Mínima	Baixa	Alta (tempdb)	Alta (keyset em tempdb)	Baixa (mas recomputa)
Locks	Mínimos	Baixos	Nenhum (snapshot)	Baixos	Altos
Scroll (FIRST/LAST/PRIOR)	✗ Não	✗ Não	✓ Sim	✓ Sim	✓ Sim
Recomendado para	Iteração simples quando cursor é inevitável	Leitura única	Auditoria (snapshot consistente)	Update de Membership fixo	Raro (custo alto)
Quando usar	90% dos casos que precisam cursor	Caso raro	Auditoria, snapshot	Update condicional	Quase nunca

◆ FAST_FORWARD é o cursor mais performático. STATIC é o mais seguro (snapshot consistente). DYNAMIC é o mais caro — evitar.

SINTAXE

Anatomia do DECLARE CURSOR

Um cursor é composto por DECLARE (definição), ciclo OPEN/FETCH/CLOSE/DEALLOCATE (uso), e controle via @@FETCH_STATUS. Cada etapa tem regras específicas.

✦ COMPONENTES DO DECLARE

- ›DECLARE nome CURSOR — identificador
- ›[LOCAL|GLOBAL] — escopo (default GLOBAL)
- ›[FORWARD_ONLY|SCROLL] — direção
- ›[STATIC|KEYSET|DYNAMIC|FAST_FORWARD] — tipo
- ›[READ_ONLY|SCROLL_LOCKS|OPTIMISTIC] — modo de lock
- ›FOR SELECT ... — consulta base
- ›[FOR UPDATE [OF colunas]] — permite updates posicionais

✦ VARIÁVEL CRÍTICA @@FETCH_STATUS

0 sucesso	**-1** fim do result set	**-2** linha removida (KEYSET)

◆ BOAS PRÁTICAS

- ›Sempre CLOSE e DEALLOCATE (mesmo em erro)
- ›Tipar variáveis locais igual às colunas
- ›Documentar motivo do cursor
- ›Considerar SET NOCOUNT ON

⚠ FINALLY GARANTE FECHAMENTO
Sem TRY/CATCH com bloco de finalização, um erro no meio do loop deixa o cursor aberto — causando vazamento de recursos e locks eternos em produção.

✦usp_IterarClientes.sql · T-SQL

```sql
-- Anatomia completa
USE
GO

CREATE OR ALTER PROCEDURE usp_IterarClientes
AS
BEGIN
   SET NOCOUNT ON
    -- 1. Variáveis locais
    DECLARE @ClienteId INT
    DECLARE @Nome  VARCHAR 100
    DECLARE @Saldo  DECIMAL 18 2
    -- 2. DECLARE CURSOR (FAST_FORWARD = recomendado)
    DECLARE           CURSOR
        LOCAL
       FAST_FORWARD
        READ_ONLY
    FOR
        SELECT
        FROM
        WHERE        'ATIVO'
        ORDER BY
    -- 3. Variável para checar sucesso
    DECLARE @Contador INT  0
    BEGIN TRY
        -- 4. OPEN — executa SELECT, popula result set
        OPEN
        -- 5. FETCH NEXT — lê primeira linha
        FETCH NEXT FROM
            INTO @ClienteId @Nome @Saldo
        -- 6. Loop enquanto houver linhas
        WHILE @@FETCH_STATUS 0
        BEGIN
            SET @Contador  1
            -- Lógica de processamento aqui
            PRINT CAST @Contador AS VARCHAR
                  ': ' @Nome
                  ' — Saldo: ' CAST @Saldo AS VARCHAR
            -- Avança para próxima linha
            FETCH NEXT FROM
                INTO @ClienteId @Nome @Saldo
        END
        PRINT 'Processados ' CAST @Contador AS VARCHAR   ' clientes'
    END TRY
   BEGIN CATCH
        PRINT 'Erro: ' ERROR_MESSAGE
    END CATCH
    FINALLY
        -- 7. SEMPRE fechar e desalocar
        IF CURSOR_STATUS local ''cur_Clientes'  0
        BEGIN
            CLOSE               -- libera locks
          DEALLOCATE             -- remove cursor
        END
   END FINALLY
END
GO
```

CICLO DE VIDA

OPEN, FETCH, CLOSE, DEALLOCATE

Quatro comandos controlam o ciclo de vida do cursor. Cada um tem papel específico — esquecer qualquer um causa bugs difíceis de rastrear.

COMANDOS

- `OPEN nome` — executa SELECT, popula result set, posiciona antes da 1ª linha
- `FETCH NEXT FROM nome INTO @vars` — lê próxima linha
- `FETCH PRIOR|FIRST|LAST|ABSOLUTE n|RELATIVE n` — para SCROLL cursors
- `CLOSE nome` — fecha result set, libera locks (pode reabrir)
- `DEALLOCATE nome` — remove cursor da memória (não pode reabrir)

ESTADOS DO CURSOR

1. **Declarado** — após DECLARE, antes de OPEN
2. **Aberto** — após OPEN, antes de CLOSE
3. **Fechado** — após CLOSE, antes de DEALLOCATE (pode reabrir)
4. **Desalocado** — após DEALLOCATE (cessou de existir)

@@FETCH_STATUS

- 0 = FETCH bem-sucedido
- 1 = FETCH além do fim do result set
- 2 = linha removida (KEYSET) — pular

⚠ PADRÃO DE BUG CLÁSSICO

- @@FETCH_STATUS é **GLOBAL por sessão**
- SP aninhada com cursor sobrescreve o status
- Testar **IMEDIATAMENTE** após FETCH

CICLO_COMPLETO.SQL — T-SQL · SQL Server

```
-- Demonstração completa do ciclo
USE VendasDB
GO

DECLARE @Id INT, @Nome VARCHAR(100)

-- 1. DECLARE
DECLARE CURSOR
    LOCAL FAST_FORWARD READ_ONLY
    FOR
    SELECT
    FROM dbo.Clientes
    WHERE     'ATIVO'
    ORDER BY

-- 2. OPEN
OPEN
PRINT 'Cursor aberto. Linhas: ' + CAST(@@CURSOR_ROWS AS VARCHAR

-- 3. FETCH primeira linha
FETCH NEXT FROM @Id, @Nome
PRINT 'Primeira linha: ' + @Nome

-- 4. WHILE
WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT 'Processando: ' + @Nome
    FETCH NEXT FROM @Id, @Nome
END

-- 5. CLOSE (libera locks, mantém definição)
CLOSE
PRINT 'Cursor fechado.'

-- 6. DEALLOCATE (remove da memória)
DEALLOCATE
PRINT 'Cursor desalocado.'
GO

-- Cursor SCROLL: navegação bidimensional
DECLARE SCROLL CURSOR
    LOCAL STATIC READ_ONLY
    FOR
    SELECT
    FROM dbo.Clientes
    ORDER BY

OPEN

-- Navegação completa
FETCH FIRST FROM -- primeira linha
FETCH LAST FROM  -- ultima linha
FETCH PRIOR FROM -- anterior
FETCH NEXT FROM -- próxima
FETCH ABSOLUTE 5 FROM      -- 5ª linha
FETCH RELATIVE 3 FROM      -- 3 a frente da atual
FETCH RELATIVE -2 FROM     -- 2 atrás da atual

CLOSE
DEALLOCATE
GO
```

CONTROLE DE FLUXO

@@FETCH_STATUS
e Controle de Loop

`@@FETCH_STATUS` é a variável global que indica o resultado do último FETCH. É o mecanismo padrão de controle de loop em cursors — mas tem armadilhas importantes.

VALORES

- 0 — FETCH bem-sucedido, linha disponível
- 1 — FETCH falhou (além do fim)
- 2 — linha removida (apenas em KEYSET)

CARACTERÍSTICAS CRÍTICAS

- **Global por sessão** — compartilhada entre SPs aninhadas
- Atualizada por qualquer FETCH na sessão
- Deve ser testada **IMEDIATAMENTE** após FETCH
- Não persiste entre execuções

PADRÕES PROFISSIONAIS

- Guardar status em variável local antes de testar
- Tratar -2 (linha removida) — pular
- Documentar motivo do cursor
- Considerar `WHILE` vs `UNTIL`

ANTI-PATTERNS

- WHILE @@FETCH_STATUS = 0 sem FETCH inicial
- SPs aninhadas com cursor sobrescrevendo status
- FETCH no início E no fim do loop (redundante)

Variável local auxiliar

```
DECLARE @Status INT = 0
FETCH NEXT FROM  INTO @vars
SET @Status = @@FETCH_STATUS
WHILE @Status = 0 BEGIN
  FETCH  SET @Status = @@FETCH_STATUS
END
```

USP_PROCESSARCOMCURSORSEGURO.SQL — T-SQL · SQL Server

```
-- Padrão CORRETO: armazenar em variável local
CREATE OR ALTER PROCEDURE usp_ProcessarComCursorSeguro
AS
BEGIN
 SET NOCOUNT ON

  DECLARE @Id INT, @Nome VARCHAR(100), @Status INT

  DECLARE CURSOR
      LOCAL FAST_FORWARD READ_ONLY
     FOR SELECT  FROM dbo.Clientes
     WHERE  'ATIVO'
     ORDER BY

   OPEN

    -- FETCH inicial
    FETCH NEXT FROM @Id, @Nome
    SET @Status = @@FETCH_STATUS  -- captura imediatamente

    -- Loop controlado por variável local
   WHILE @Status = 0
   BEGIN
      -- Processa linha atual
      PRINT 'Processando cliente ' + CAST(@Id AS VARCHAR)
          + ': ' + @Nome

      -- Lógica de negócio aqui
      UPDATE dbo.Clientes
      SET  GETDATE()
      WHERE  @Id

      -- Próximo FETCH
      FETCH NEXT FROM @Id, @Nome
      SET @Status = @@FETCH_STATUS
   END

    -- Trata linha removida (KEYSET)
   IF @Status = -2
      PRINT 'Aviso: uma linha foi removida durante a iteração'

  CLOSE
 DEALLOCATE
END
GO

-- ANTI-PATTERN 1: WHILE sem FETCH inicial
-- ✗ Não funciona — primeira iteração falha
/*
DECLARE cur CURSOR FOR SELECT Id FROM Tabela;
OPEN cur;
WHILE @@FETCH_STATUS = 0  -- ainda -1 da sessão anterior!
BEGIN
 FETCH NEXT FROM cur INTO @Id;
 -- processa
END
*/

-- ANTI-PATTERN 2: SPs aninhadas sobrescrevem @@FETCH_STATUS
-- ✗ SP_A tem cursor, chama SP_B que também tem cursor
-- @@FETCH_STATUS de SP_A é sobrescrito por SP_B!
-- Solução: SP_B retorna result set, SP_A itera com cursor único
```

SCROLL CURSORS

Parâmetros de Scroll

Cursors SCROLL permitem navegação bidimensional — primeira, última, anterior, próxima, posição absoluta e relativa. Úteis para auditoria, paginação e relatórios com navegação livre.

TIPOS QUE SUPORTAM SCROLL

STATIC KEYSET DYNAMIC

STATIC — snapshot consistente · **KEYSET** — membership fixo · **DYNAMIC** — recomputa a cada FETCH

COMANDOS DISPONÍVEIS

`FETCH FIRST`	primeira linha
`FETCH LAST`	última linha
`FETCH PRIOR`	anterior
`FETCH NEXT`	próxima (padrão)
`FETCH ABSOLUTE n`	n-ésima linha (n positivo do início, negativo do fim)
`FETCH RELATIVE n`	n à frente (+) ou atrás (-) da atual

CASOS DE USO

- Paginação manual (raro em produção)
- Auditoria — revisitar linhas
- Relatório com navegação livre
- Cálculos com janelas deslizantes

LIMITAÇÕES

- Não disponível em `FAST_FORWARD`
- Mais caro que `FORWARD_ONLY`
- Requer STATIC ou KEYSET p/ consistência

USP_NAVEGARCLIENTESSCROLL.SQL — T-SQL · SQL Server

```sql
-- Cursor SCROLL para navegação bidimensional
CREATE OR ALTER PROCEDURE usp_NavegarClientesScroll
    @Operacao VARCHAR(20)  -- 'FIRST', 'LAST', 'NEXT', 'PRIOR', 'ABSOLUTE', 'RELATIVE'
    @Offset INT = 0
AS
BEGIN
    SET NOCOUNT ON

    -- Cursor STATIC: snapshot consistente
    DECLARE cScroll CURSOR
        LOCAL STATIC READ_ONLY
        FOR
            SELECT Id, Nome, Saldo
            FROM dbo.Clientes
            WHERE Ativo = 'ATIVO'
            ORDER BY Id

    DECLARE @Id INT, @Nome VARCHAR(100), @Saldo DECIMAL(18,2)

    OPEN cScroll

    -- Navegação baseada no parâmetro
    IF @Operacao = 'FIRST'
        FETCH FIRST FROM cScroll INTO @Id, @Nome, @Saldo
    ELSE IF @Operacao = 'LAST'
        FETCH LAST FROM cScroll INTO @Id, @Nome, @Saldo
    ELSE IF @Operacao = 'NEXT'
        FETCH NEXT FROM cScroll INTO @Id, @Nome, @Saldo
    ELSE IF @Operacao = 'PRIOR'
        FETCH PRIOR FROM cScroll INTO @Id, @Nome, @Saldo
    ELSE IF @Operacao = 'ABSOLUTE'
        FETCH ABSOLUTE @Offset FROM cScroll INTO @Id, @Nome, @Saldo
    ELSE IF @Operacao = 'RELATIVE'
        FETCH RELATIVE @Offset FROM cScroll INTO @Id, @Nome, @Saldo

    IF @@FETCH_STATUS = 0
        SELECT @Id AS Id, @Nome AS Nome, @Saldo AS Saldo
    ELSE
        PRINT 'Operação falhou (posição inválida ou fim do result set)'

    CLOSE cScroll
    DEALLOCATE cScroll
END
GO

-- Exemplo: paginação manual com SCROLL
DECLARE cScroll CURSOR
    LOCAL STATIC READ_ONLY
    FOR
    SELECT Id, Nome
    FROM dbo.Clientes
    ORDER BY Id

DECLARE @Id INT, @Nome VARCHAR(100), @Pagina INT = 1

OPEN cScroll

-- Página 1: linhas 1 a 10
FETCH ABSOLUTE 1 FROM cScroll INTO @Id, @Nome
PRINT 'Página ' + CAST(@Pagina AS VARCHAR) + ':'
WHILE @@FETCH_STATUS = 0 AND @Pagina <= 10
BEGIN
    PRINT '  ' + CAST(@Id AS VARCHAR) + ' - ' + @Nome
    FETCH NEXT FROM cScroll INTO @Id, @Nome
    SET @Pagina = @Pagina + 1
END

CLOSE cScroll
DEALLOCATE cScroll
GO
```

EXEMPLO PRÁTICO

Cursor em Stored Procedure

Exemplo real: SP que processa pedidos pendentes um a um, enviando cada um para sistema externo (logística) via execução de SP. Cenário legítimo de cursor — integração com sistema externo.

CENÁRIO LEGÍTIMO

Cada pedido exige resposta individual da logística — não há alternativa set-based direta.

RECURSOS DO EXEMPLO

- ⚡Cursor `FAST_FORWARD` (otimizado)
- ✦Variáveis locais para FETCH
- ⛨TRY/CATCH com `FINALLY` (garante CLOSE/DEALLOCATE)
- ▦`@@FETCH_STATUS` em variável local
- ▤Log estruturado por item
- ⊗Tratamento de erro individual (não aborta lote)
- Contador de progresso
- ▤Documentação via `comment`

95%
CASOS TÊM ALTERNATIVA SET-BASED

5%
INTEGRAÇÃO EXTERNA JUSTIFICA CURSOR

✦ USP_PROCESSARPEDIDOSPENDENTES.SQ

T-SQL · SQL Server

```
-- SP: processa pedidos pendentes enviando para logística
CREATE OR ALTER PROCEDURE dbo.usp_ProcessarPedidosPendentes
AS
BEGIN
SET NOCOUNT ON
    -- Variáveis locais
DECLARE @PedidoId INT
DECLARE @ClienteId INT
DECLARE @ValorTotal DECIMAL(18,2)
DECLARE @Processados INT = 0
DECLARE @Erros INT = 0
    -- Cursor FAST_FORWARD: melhor performance
DECLARE cPedidos CURSOR
    LOCAL FAST_FORWARD READ_ONLY
    FOR
    SELECT
      FROM dbo.Pedidos
     WHERE Status = 'PENDENTE'
     ORDER BY
BEGIN TRY
    OPEN
      -- FETCH inicial
      FETCH NEXT FROM
        @PedidoId, @ClienteId, @ValorTotal
      SET @Status = @@FETCH_STATUS
    WHILE
    BEGIN
    BEGIN TRY
      -- Marca como EM_PROCESSAMENTO
      UPDATE dbo.Pedidos
        SET Status = 'EM_PROCESSAMENTO'
        WHERE PedidoId = @PedidoId
      -- Chama API externa (simulada)
      EXEC dbo.usp_EnviarParaLogistica
        @PedidoId = @PedidoId,
        @ClienteId = @ClienteId,
        @ValorTotal
      -- Marca como ENVIADO
      UPDATE dbo.Pedidos
        SET Status = 'ENVIADO',
            DataEnvio = GETDATE()
        WHERE PedidoId = @PedidoId
      -- Log de sucesso
      INSERT INTO dbo.LogProcessamento
        (PedidoId, DataLog, Mensagem)
      VALUES
        (@PedidoId, GETDATE(),
         'Enviado para logística')
      SET @Processados = @Processados + 1
    END TRY
    BEGIN CATCH
      -- Erro individual: loga mas NÃO aborta o lote
      INSERT INTO dbo.LogProcessamento
        (PedidoId, DataLog, Mensagem)
      VALUES
        (@PedidoId, GETDATE(),
         'ERRO ao enviar para logística: '
         + ERROR_MESSAGE())
      -- Rollback do status
      UPDATE dbo.Pedidos
        SET Status = 'PENDENTE'  -- volta para fila
        WHERE PedidoId = @PedidoId
      SET @Erros = @Erros + 1
    END CATCH
    -- Próximo pedido
    FETCH NEXT FROM
      @PedidoId, @ClienteId, @ValorTotal
    SET @Status = @@FETCH_STATUS
  END
  PRINT 'Processamento concluído.'
  PRINT 'Processados: ' + CAST(@Processados AS VARCHAR)
  PRINT 'Erros: ' + CAST(@Erros AS VARCHAR)
END TRY
BEGIN CATCH
  -- Erro fatal: loga e re-lança
  THROW
END CATCH
FINALLY
  -- SEMPRE fecha e desaloca
  IF CURSOR_STATUS('local', 'cPedidos') >= 0
  BEGIN
    CLOSE cPedidos
    DEALLOCATE cPedidos
  END
END FINALLY
GO
```

BLOCO 6.2

6.2

Navegação, Desempenho e Alternativas

Por que cursors são problemáticos em produção, como medir e mitigar seu impacto, e quais alternativas set-based o SQL Server oferece para substituí-los na maioria dos cenários.

01 ⚠ Problemas dos cursors

02 ✦ Desempenho e locks

03 ⇄ Alternativas set-based

CONFRONTO CENTRAL

CURSOR × SET-BASED — quando usar um ou outro?

BLOCO 6.2 · PROBLEMAS

Por que Cursors são Problemáticos

6
PROBLEMAS CENTRAIS

Seis razões técnicas pelas quais cursors devem ser evitados em produção — e por que a regra profissional é "comece pelo set-based, justifique cursor caso a caso".

01

Não set-based

Operam linha-a-linha, ignorando as otimizações do engine relacional — não aproveitam paralelismo, índices em massa nem álgebra relacional.

1×N ITERAÇÕES EM VEZ DE 1 ROUND-TRIP

02

Lentidão

Até [redacted] mais lento que [redacted] / [redacted] set-based em tabelas médias — o custo cresce linearmente com o número de linhas.

100× MAIS LENTO EM LARGA ESCALA

03

Locks prolongados

Mantêm locks durante toda a iteração — da abertura do [redacted] até o [redacted]. Quanto maior o loop, maior a janela de bloqueio sobre a tabela.

Δt BLOQUEIO CRESCE COM O LOOP

04

Tempdb overhead

Cursors [redacted] e [redacted] materializam o result set em [redacted] — saturam I/O e competem com outras sessões por espaço temporário.

tempdb MATERIALIZAÇÃO EM DISCO

05

Concorrência prejudicada

Bloqueiam escrita paralela na tabela de origem — writers esperam, throughput da aplicação cai, contencão aumenta sob carga alta.

↓ THROUGHPUT DA APLICAÇÃO

06

Bugs difíceis de rastrear

[redacted] é global por sessão — SPs aninhadas com cursor sobrescrevem o status. Resultado: loops silenciosos, race conditions, comportamento não-determinístico.

@global ESTADO COMPARTILHADO ENTRE SPS

REGRA PROFISSIONAL Em **95% dos casos** há alternativa set-based. Cursor só se justifica em integração com sistema externo, fila ou processamento genuinamente row-by-row.

IMPACTO MENSURÁVEL

Custos de CPU, Memória e Locks

Cada cursor adiciona custo mensurável: CPU por linha, memória para o result set, locks estendidos. Em sistemas de alta carga, o impacto é direto e mensurável via DMVs.

CUSTOS TÍPICOS

- CPU: ~1ms por FETCH (overhead fixo)
- Memória: STATIC/KEYSET criam cópia em tempdb
- Locks: mantidos durante toda a iteração
- TempDB: STATIC/KEYSET consomem espaço
- Network: SP remota ⇒ round-trip por FETCH

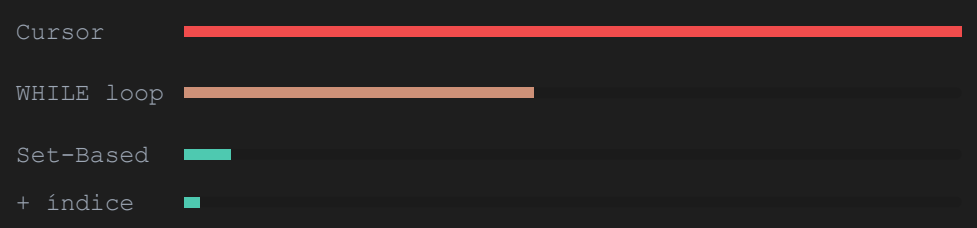
PADRÕES DE OTIMIZAÇÃO (CURSOR INEVITÁVEL)

- Sempre `FAST_FORWARD` (mais performático)
- `READ_ONLY` quando não atualiza
- `LOCAL` (não GLOBAL)
- Minimizar lógica dentro do loop
- FETCH só colunas necessárias (não `SELECT *`)
- Manter loop curto

DMVS PARA DIAGNÓSTICO

- `sys.dm_exec_cursors` — cursors ativos
- `sys.dm_os_waiting_tasks` — sessões esperando
- `sys.dm_tran_locks` — locks mantidos por cursor

MÉTRICAS · 10.000 LINHAS




```
-- 1. Inspecionar cursors ativos
SELECT
          AS
                    1000 AS
                     1000 AS
          AS
 FROM                0    -- 0 = todas as sessões
CROSS APPLY
 ORDER BY            DESC
 GO
```

```
-- 2. Cursor vs set-based · desconto 10% VIP

-- ❌ ANTI-PATTERN: Cursor (lento)
DECLARE    CURSOR LOCAL FAST_FORWARD READ_ONLY
   FOR SELECT
       FROM          WHERE        'VIP'
DECLARE@IdINT  @SaldoDECIMAL18 2
OPEN
FETCH NEXT FROM  INTO@Id @Saldo
WHILE              0
BEGIN
   UPDATE          SET       @Saldo  0.9
    WHERE          @Id
    FETCH NEXT FROM  INTO@Id @Saldo
END
CLOSE    DEALLOCATE
-- Tempo: ~10s p/ 10.000 clientes

-- ✅ PADRÃO CORRETO: Set-based UPDATE
UPDATE          SET              0.9
WHERE       'VIP'
-- Tempo: ~50ms p/ 10.000 (200x mais rápido!)
GO
```

```
-- 3. WHILE loop como alternativa procedural
DECLARE@IdINT  1
WHILE@Id   SELECTMAX          FROM
BEGIN
   UPDATE
    SET              0.9
    WHERE          @IdAND        'VIP'
    SET@Id  1
END
-- Tempo: ~5s (row-by-row sem cursor overhead)
GO
```

```
-- 4. Locks mantidos por cursor
SELECT

   OBJECT_NAME                                AS
                    1000 AS
FROM
LEFT JOIN
   ON
WHERE                  IN
   SELECT          FROM                  0

ORDER BY
GO
```

WHILE Loop vs Cursor vs Set-Based

Três abordagens para processamento iterativo — quando cada uma se justifica

Evitar Cuidado Recomendado

Aspecto	✕Cursor	✦WHILE Loop	✦Set-Based (UPDATE/SELECT)
Paradigma	Procedural (row-by-row)	Procedural (counter-based)	Declarativo (conjunto)
Performance	✗ Pior (overhead FETCH)	⚠ Médio (sem overhead FETCH)	✓✓ Melhor (1 operação)
Código	Verboso (DECLARE/OPEN/FETCH/CLOSE/DEALLOCATE)	Conciso (DECLARE/WHILE/SET)	Mínimo (1 statement)
Flexibilidade	Alta (pode parar, pular, retroceder)	Média (counter simples)	Baixa (tudo ou nada)
Controla fluxo?	✓ Sim (FIRST/LAST/PRIOR)	✗ Não (só counter)	✗ Não (atômico)
Recursos	Result set explícito	Acesso por PK	Operação em massa
Locks	Estendidos (todo o loop)	Por iteração (curtos)	Único lock X por transação
Memória	Cursor + result set	Mínima	Mínima (apenas buffer)
Paralelismo	✗ Não	✗ Não	✓ Sim (SQL Server 2016+)
Usa quando	Integração externa, navegação bidimensional	Counter simples, REBUILD/REORG por índice	TODA operação em massa (default)
Caso típico	Enviar pedidos para logística	Manutenção de índices	UPDATE com WHERE, INSERT SELECT, MERGE
Recomendado	✗ Raramente	⚠ Em operações admin	✓ Sempre que possível

◆ Regra prática: 1 Tente set-based. 2 Se impossível, tente WHILE. 3 Só use cursor se precisar navegação bidimensional ou integração externa.

ALTERNATIVA 1/4

UPDATE com CASE (Set-Based)

Quando a lógica do cursor é *"atualizar cada linha de forma diferente baseada em uma condição"*, o CASE resolve tudo em um único UPDATE — **100x+ mais rápido**.

PADRÃO

- `UPDATE tabela SET col = CASE WHEN cond1 THEN v1 WHEN cond2 THEN v2 ELSE v3 END`
- Subqueries permitidas no CASE
- Combina com JOIN para condições complexas
- Múltiplas colunas: `SET a=CASE…, b=CASE…`

VANTAGENS

- ✓ 1 operação atômica · paralelizável
- ✓ Sem overhead de FETCH · sem locks estendidos
- ✓ Código mais conciso

CASOS DE USO

- Aplicar desconto diferenciado por faixa
- Atualizar status baseado em data
- Calcular totais baseado em condição
- Sincronizar com tabela externa via JOIN

✕ ANTI-PATTERN → ✅ PADRÃO CORRETO

```
-- ✕ ANTI-PATTERN: Cursor para atualizar clientes
DECLARE   CURSOR LOCAL FAST_FORWARD
  FOR SELECT          FROM
DECLARE @Id INT @Faixa VARCHAR(20)
OPEN
 FETCH NEXT FROM INTO @Id @Faixa
WHILE          0
BEGIN
   IF @Faixa 'VIP' UPDATE        SET          0.9 WHERE       @Id
   ELSE IF @Faixa 'GOLD' UPDATE        SET          0.95 WHERE       @Id
   ELSE UPDATE        SET          0.98 WHERE       @Id
    FETCH NEXT FROM INTO @Id @Faixa
END
CLOSE  DEALLOCATE
 -- Tempo: ~12 segundos para 10.000 clientes
 GO

 -- ✅ PADRÃO CORRETO: UPDATE com CASE
UPDATE
 SET
   CASE
      WHEN      'VIP' THEN      0.9
      WHEN      'GOLD' THEN      0.95
      ELSE      0.98
   END
 -- Tempo: ~30ms para 10.000 clientes (400x mais rápido!)
 GO
```

JOIN + MÚLTIPLAS COLUNAS

```
 -- Caso mais complexo: com JOIN e múltiplas colunas
UPDATE
 SET

              GETDATE
          CASE
      WHEN                    100 THEN 'BLOQUEADO'
      ELSE
   END
 FROM
INNER JOIN
    ON
 WHERE       'ATIVO'
 GO
```

SUBQUERY · NOT EXISTS

```
 -- Atualização condicional com subquery
UPDATE
 SET     'ATRASADO'
           GETDATE
 WHERE     'PENDENTE'
  AND DATEDIFF DAY          GETDATE    7
AND NOT EXISTS
    SELECT 1 FROM
     WHERE
       AND       'CONFIRMADO'

 GO
```

ALTERNATIVA 2/4

CTEs Recursivas e Window Functions

Quando o cursor precisa *"olhar a linha anterior"* ou processar hierarquias, CTEs recursivas e window functions resolvem de forma **set-based e elegante**.

CTE RECURSIVA

- Membro âncora (caso base)
- `UNION ALL` + membro recursivo
- Referencia a própria CTE
- Limite default: 100 níveis (`OPTION MAXRECURSION`)
- Hierarquias, BOM, organogramas

WINDOW FUNCTIONS

`LAG(col, n) / LEAD(col, n)`
valor de n linhas atrás / à frente

`ROW_NUMBER() OVER (ORDER BY …)`
número sequencial — paginação

`RANK() / DENSE_RANK()`
ranking com ou sem gaps

`SUM/AVG/MIN/MAX OVER (PARTITION BY …)`
agregados em janela móvel

CASOS TÍPICOS

- Hierarquia de funcionários (CEO → gerentes → subordinados)
- BOM (Bill of Materials)
- Saldo acumulado em extrato bancário
- Top N por categoria

CTE RECURSIVA · HIERARQUIA

```
-- ✗ ANTI-PATTERN: Cursor para hierarquia
-- (código omitido, mas seria ~30 linhas)

-- ✓ CTE RECURSIVA: hierarquia de funcionários
WITH  AS
    -- Âncora: presidente (sem chefe)
   SELECT  AS
    FROM
    WHERE     IS NULL

   UNION ALL

    -- Recursivo: subordinados
   SELECT
    FROM
   INNER JOIN       ON

SELECT
 FROM
 WHERE
ORDER BY
 GO
```

ROW_NUMBER · PAGINAÇÃO

```
-- ✓ ROW_NUMBER: paginação sem cursor
SELECT
 FROM
   SELECT
     ROW_NUMBER OVER ORDER BY      AS
    FROM
    WHERE     'ATIVO'

WHERE     BETWEEN AND    -- página 1
 -- Próxima página: BETWEEN 11 AND 20
 GO
```

SUM OVER · SALDO ACUMULADO

```
-- ✗ ANTI-PATTERN: Cursor para saldo acumulado
-- (processa cada transação em ordem, soma ao saldo)

-- ✓ WINDOW FUNCTION: saldo acumulado em extrato
SELECT


    SUM     OVER
       ORDER BY
        ROWS BETWEEN UNBOUNDED PRECEDING
                 AND CURRENT ROW
      AS
 FROM
 WHERE
 ORDER BY
 GO
```

LAG/LEAD · COMPARAR LINHA

```
-- ✓ LAG/LEAD: comparar com linha anterior
SELECT


         LAG       OVER
     ORDER BY
    AS
   ROUND

        100.0  LAG       OVERORDER BY
               LAG       OVERORDER BY

    AS
 FROM
 ORDER BY
 GO

-- ✓ Top N por categoria
SELECT
 FROM
   SELECT
    -- (continua: ROW_NUMBER() OVER (PARTITION BY Categoria
```

ALTERNATIVA 3/4 E 4/4

MERGE e Alternativas Modernas

MERGE substitui cursor para *upsert* (insert-or-update). **CLR Integration** permite linguagens .NET para loops complexos. **Service Broker** oferece fila assíncrona. **JSON** permite iterar arrays em set-based.

MERGE (UPSERT)

- Combina INSERT + UPDATE + DELETE em 1 comando
- Linha existe: UPDATE · não existe: INSERT
- Existe na origem mas não no alvo: DELETE (opcional)
- Substitui cursor *"verifica se existe, atualiza ou insere"*

ALTERNATIVAS MODERNAS

CLR Integration — SQL 2005+

SPs em C#/VB.NET compiladas para DLL — útil para regex, cálculos complexos, loops pesados. Não é set-based, mas executa mais rápido que T-SQL procedural.

Service Broker — Nativo

Fila assíncrona nativa. Trigger/procedure enfileira mensagem; procedure separada processa. **Desacopla produtor e consumidor.**

JSON manipulation — SQL 2016+

OPENJSON expande array em tabela · JSON_VALUE extrai campos · set-based com dados estruturados.

✗ ANTI-PATTERN

```
-- ✗ ANTI-PATTERN: Cursor para UPSERT
DECLARE  CURSOR FOR SELECT  FROM
DECLARE @Id INT @Nome VARCHAR(100)
OPEN
FETCH NEXT FROM INTO @Id @Nome
WHILE                0
BEGIN
   IF EXISTS(SELECT 1 FROM        WHERE        @Id
      UPDATE          SET      @Nome WHERE        @Id
   ELSE
      INSERT                        VALUES(@Id @Nome
    FETCH NEXT FROM INTO @Id @Nome
END
CLOSE  DEALLOCATE
GO
```

JSON · ARRAY SEM CURSOR

```
-- ✓ JSON: iterar array sem cursor
DECLARE @Json NVARCHAR(MAX)  N'[
 {"Id": 1, "Nome": "João"},
 {"Id": 2, "Nome": "Maria"},
 {"Id": 3, "Nome": "Pedro"}
]'

-- OPENJSON expande array como tabela
SELECT
  JSON_VALUE     '$.Id' AS
  JSON_VALUE     '$.Nome' AS
FROM OPENJSON(@Json)

-- INSERT set-based a partir de JSON
INSERT  INTO
SELECT
  JSON_VALUE     '$.Id'
  JSON_VALUE     '$.Nome'
FROM OPENJSON(@Json)
GO
```

✓ MERGE · UPSERT

```
-- ✓ MERGE (UPSERT em um comando)
MERGE  INTO         AS
 USING              AS
   ON
  WHEN MATCHED THEN
   UPDATE SET
                                GETDATE
  WHEN NOT MATCHED BY TARGET THEN
    INSERT
    VALUES                      GETDATE
  WHEN NOT MATCHED BY SOURCE THEN
    DELETE  -- opcional: deleta quem sumiu
 GO
```

CONCEITOS · ARQUITETURA

```
-- ✓ Service Broker: fila assíncrona (conceito)
-- Em vez de cursor que processa item por item:
-- 1. Procedure/Trigger: INSERT na tabela de fila
-- 2. Job/Activation procedure: lê fila e processa
-- 3. Itens processados de forma assíncrona
-- Vantagem: produtor não espera processamento
-- Substitui cursor síncrono em integrações externas
```

CONCEITOS · PERFORMANCE

```
-- ✓ CLR Integration (conceito)
-- Em vez de cursor T-SQL lento:
-- CREATE ASSEMBLY MinhaLib FROM 'MinhaLib.dll'
-- CREATE PROCEDURE usp_Processar
-- AS EXTERNAL NAME MinhaLib.StoredProcedures.Processar
-- C# compilado: 10-100x mais rápido que T-SQL procedural
-- Útil para: regex, cálculos complexos, loops pesados
```

✓ Checklist Profissional de Cursores

Antes de escrever cursor, pergunte-se — *imprima e cole na parede*

1–6 · TENTE ALTERNATIVAS PRIMEIRO

1. ✓ Já tentei set-based? UPDATE com CASE
2. ✓ Já tentei CTE recursiva? para hierarquias, BOM, organogramas
3. ✓ Já tentei window function? LAG / LEAD / ROW_NUMBER / SUM OVER
4. ✓ Já tentei MERGE? para UPSERT (insert-or-update)
5. ✓ Já tentei OPENJSON? para iterar arrays sem cursor
6. ✓ Já tentei WHILE loop? última opção procedural (counter)

7–10 · IMPLEMENTAÇÃO CORRETA (CURSOR INEVITÁVEL)

7. ✓ FAST_FORWARD — opção mais performática (read-only forward-only)
8. ✓ READ_ONLY — quando não atualiza via cursor
9. ✓ LOCAL — nunca GLOBAL — escopo controlado
10. ✓ TRY / CATCH — tratamento de erros e DEALLOCATE seguro

11–12 · OPERAÇÃO EM PRODUÇÃO

11. ✓ Monitore via DMV — sys.dm_exec_cursors · sys.dm_tran_locks
12. ✓ Documente via SP — sp_addextendedproperty — fácil identificação

O que vimos hoje

6.1 CRIAÇÃO E MANIPULAÇÃO

Conceito · 5 tipos de cursor (FAST_FORWARD mais performático, STATIC para auditoria, DYNAMIC evitar) · anatomia DECLARE/OPEN/FETCH/CLOSE/DEALLOCATE · @@FETCH_STATUS e suas armadilhas · parâmetros de scroll (FIRST/LAST/PRIOR/ABSOLUTE/RELATIVE) · exemplo prático em SP com integração externa.

6.2 DESEMPENHO E ALTERNATIVAS

Por que cursors são problemáticos (paradigma set-based, CPU, locks, vazamento) · WHILE loop vs cursor vs set-based · alternativas: UPDATE com CASE, CTEs recursivas, window functions, MERGE, CLR, Service Broker, JSON.

Próximos passos · Desafio

```
-- Desafio para Aula 07:
-- 1. Identifique cursor existente em produção
-- 2. Refatore para set-based (CASE/CTE/window)
-- 3. Meça tempo antes/depois (estimado 100x+ ganho)
-- 4. Se cursor inevitável, aplique FAST_FORWARD + TRY/CATCH
-- 5. Documente via sp_addextendedproperty
```

→ Aula 07: Otimização Avançada · Planos de Execução · Índices · Estatísticas · Query Store